

Some Basics in R

Christopher Hare

1.1 RStudio

RStudio provides a more friendly environment for working in R.

You can run RStudio from the cloud here: <https://rstudio.cloud/>. This is the easiest and recommended option for the class.

OR, if you wish, you can download RStudio here: <https://rstudio.com/products/rstudio/>

1.2 Rules for Entering Code

Commands are case sensitive: For example, the command `var` and `VAR` have two different meanings.

A simple “return” separates commands.

R is an object-oriented programming language: we create objects using arrow keys, as below:

```
test <- 3
test
```

1.3 Arithmetic Commands

Basic arithmetic operations are coded in R: for example, addition (+), subtraction (−), multiplication (*), division (/), but also exponents (^) and others.

Parentheses denote order of operations. Example:

```
(3+5/78)^3*7
```

This is a rather simple example which gives us the output:

```
[1] 201.3761
```

1.4 Random Numbers

We can generate (pseudo)random numbers in R with the following code. For instance, let's roll some dice:

```
sample(1:6, 1)
sample(1:6, 100, replace=TRUE)
sample(1:6, 100, replace=TRUE) + sample(1:6, 100, replace=TRUE)
```

Let's draw a histogram of the values and look at the means/medians using the commands:

```
rolls <- sample(1:6, 1000, replace=TRUE) + sample(1:6, 1000, replace=TRUE)
hist(rolls)
mean(rolls)
median(rolls)
```

1.5 Installing and Loading Packages

In many instances, we need to install packages of data and functions into R. On first use, we need to install:

```
install.packages("car")
```

Note that a pop-up window will prompt you to choose a server. For faster results, choose the most geographically proximate. On subsequent uses, we need only load a package with the library command:

```
library(car)
```

1.6 Loading and Manipulating Data

We can then load data from a package such as `car` package by loading that package and then using the command:

```
data(States)
```

To view the first few observations, use:

```
head(States)
```

We can access a single variable using, for example:

```
States$SATM
```

Suppose that we wanted to only examine states in our dataset with an average SATM score of more than 500. We could create this subset by typing:

```
States.highSATM <- subset(States, subset = SATM > 500)
```

We could count the number of states with average SATM scores of greater than 500 using (at least two) approaches. One is:

```
dim(States.highSATM)
```

Another way is to count the number of times a condition is met:

```
States$SATM > 500  
table(States$SATM > 500)
```

1.7 Plotting Data

Let's begin by plotting a histogram of the state average teacher salaries (which is in \$1000s):

```
hist(States$pay, xlab="This is the x-axis", main="This is the main title", ylim=c(0,20))
```

We can do a box-and-whisker plot using the command:

```
boxplot(States$pay ~ States$region)
```

Scatterplots can be generated by listing an independent and dependent variable on the x and y axis, respectively:

```
plot(States$percent, States$SATM)
```

Can add what's known as a “smoother” with the command:

```
plot(States$percent, States$SATM)  
lines(lowess(States$percent, States$SATM))  
lines(lowess(States$percent, States$SATM, f=0.1))
```

1.8 for loops

```
q <- NA  
for (i in 1:10){  
  q[i] <- i^2  
}  
q
```

1.9 Probability Distributions

This section introduces some important concepts to be discussed in the second half of the course: probability distributions. Probability distributions define the probability that a random variable takes on a certain value or falls within a range of values.

When we say “distributions,” we’re really referring to one of the major distributional families. They’re called families because we can adjust the shape of the distributions themselves in an infinite number of ways.

Here, let’s look at two common (and one weird) distributions:

```
normal <- rnorm(1000, mean=0, sd=2)
plot(density(normal))
#
uniform <- runif(1000, min=0, max=1)
plot(density(uniform))
#
cauchy <- rcauchy(1000, location=0, scale=1)
plot(density(cauchy))
#
plot(density(normal), ylim=c(0,0.5))
lines(density(cauchy), col="blue")
```